JonesAuto Database Management System

CPSC 3660

Gurmann Basran

April 8, 2026

1 Introduction

JonesAuto is a used car dealership based in Lethbridge. They buy cars at auctions, fix them up, and resell them to customers. A lot of their sales are financed, so they have to keep track of monthly payments on top of everything else. Right now they do all of this on paper, which honestly gets pretty messy once you are dealing with dozens of cars and customers at the same time.

I built a database system to replace that whole paper-based workflow. It covers the full lifecycle of a car: buying it at auction, recording what repairs it needs, selling it to a customer, setting up warranty coverage, and tracking monthly payments. We learned in class why a DBMS beats flat files: things like data redundancy, consistency problems, and not being able to query anything easily. This project is pretty much that idea applied to a real business. The system runs on PHP and MySQL on Linux, using PHP's built-in development server.

The system has three main parts: input forms for entering data, output reports for pulling summaries, and a queries page for answering specific business questions. There are 6 forms, 6 reports, and 12 queries total. Seven of those queries use advanced SQL features like JOINs, subqueries, and GROUP BY with HAVING. I tried to cover a good range of what we learned in class throughout the semester.

2 Stage 1: Overall Design

2.1 System Goals

The main goal was to replace JonesAuto's paper records with a proper database system. The system needs to track the full car lifecycle from purchase at auction all the way through to customer payments. It also needs to give employees quick access to customer info, vehicle history, and financial summaries without digging through filing cabinets.

2.2 Input Forms

Form	Purpose
Purchase Form	Records a car bought at auction with vehicle details, price, buyer, repairs
Sale Form	Records a car sale with financing, customer info, employment history
Warranty Form	Adds warranty coverage to an existing sale
Payment Form	Records monthly payments, detects late payments, updates credit stats
Customer Form	Adds new customers and shows existing customer records
Employee Form	Adds new employees (buyers/salespeople) and shows existing records

2.3 Output Reports

Report	Purpose
Current Inventory	Shows available cars on the lot, filterable by make, with repair cost totals
Sales Summary	Shows completed sales with profit calculation, filterable by date range
Payment History	Shows payment records for a selected customer, highlights late payments
Repair Costs	Compares estimated vs actual repair costs, highlights overruns
Active Warranties	Lists warranty coverage with color-coded expiry urgency
Late Payments	Shows customers with late payments, sorted worst-first

2.4 System Users

There are three types of users: buyer employees who go to auctions and fill out purchase forms, salesperson employees who handle sales and warranties and payments, and office staff who run reports and queries. The system does not have a login since it is a small dealership and everyone pretty much shares the same machine.

2.5 Initial Design Assumptions

- Cars are only acquired through auctions (though there is an `is_auction` flag for flexibility)
- Each car has exactly one purchase record and at most one sale
- Customers can buy multiple cars over time
- Warranties are tied to a specific sale, not directly to a customer
- The dealership is in Lethbridge, Alberta (all sample data uses Alberta locations)
- Payments are monthly, and "late" means `paid_date` is after `due_date`

I made these assumptions based on the business description in the project spec. Some of them I had to decide on my own since the spec did not cover every edge case.

3 Stage 2: ER Diagram and Database Schema

We covered ER diagrams in class using Chen notation, where rectangles represent entities, diamonds represent relationships, and ovals represent attributes. I designed the ER diagram before writing any SQL since I could map the ER model directly to relational tables, giving a more structured approach than jumping straight to CREATE TABLE statements.

The full ER diagram for JonesAuto is on the following page (Figure 1). The diagram shows all 10 entity sets, the 11 relationships between them, primary keys (underlined), and cardinality labels. Double lines indicate total participation (the entity must participate in the relationship), and single lines indicate partial participation.

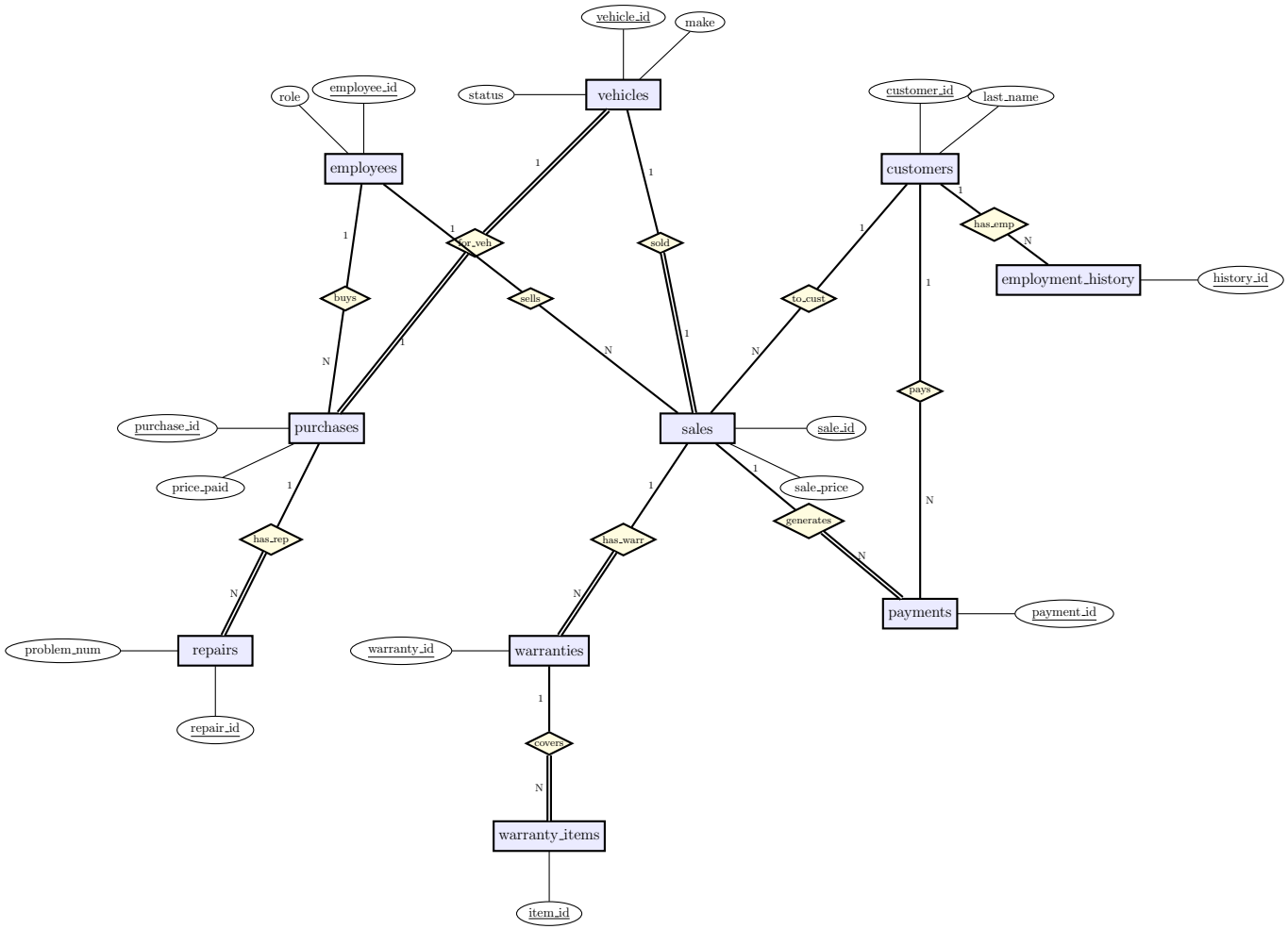


Figure 1: Entity-Relationship Diagram for JonesAuto Database (Chen Notation)

3.1 Entities and Attributes

Entity	Attributes	Primary Key
employees	employee_id, first_name, last_name, phone, role	employee_id
vehicles	vehicle_id, make, model, year, color, miles, condition_desc, book_price, style, interior_color, status	vehicle_id
purchases	purchase_id, vehicle_id, employee_id, purchase_date, location, seller_dealer, is_auction, price_paid	purchase_id
repairs	repair_id, purchase_id, problem_num, description, est_cost, actual_cost	repair_id
customers	customer_id, first_name, last_name, phone, address, city, state, zip, gender, dob, num_late_payments, avg_days_late	customer_id
employment_history	history_id, customer_id, employer, title, supervisor_phone, employer_address, start_date	history_id
sales	sale_id, vehicle_id, customer_id, employee_id, sale_date, total_due, down_payment, financed_amount, sale_price, commission	sale_id
warranties	warranty_id, sale_id, vehicle_id, customer_id, employee_id, warranty_sale_date, total_cost, monthly_cost	warranty_id
warranty_items	item_id, warranty_id, warranty_type, start_date, length_months, cost, deductible, items_covered	item_id
payments	payment_id, customer_id, sale_id, payment_date, due_date, paid_date, amount, bank_account	payment_id

3.2 Relationships

1. **employees to purchases** (one-to-many): one buyer employee can make many purchases
2. **vehicles to purchases** (one-to-one): each vehicle has exactly one purchase record
3. **purchases to repairs** (one-to-many): one purchase can have multiple repair problems
4. **customers to employment_history** (one-to-many): each customer can have multiple employer records
5. **vehicles to sales** (one-to-one): each vehicle can be sold once
6. **customers to sales** (one-to-many): one customer can buy multiple cars
7. **employees to sales** (one-to-many): one salesperson can handle many sales
8. **sales to warranties** (one-to-many): one sale can have multiple warranty contracts
9. **warranties to warranty_items** (one-to-many): each warranty can cover multiple items

10. **sales to payments** (one-to-many): one sale generates multiple monthly payments
11. **customers to payments** (one-to-many): denormalized link for quick customer payment lookup

Total: 14 foreign key constraints across the schema.

3.3 Relational Schema

Figure 2 shows the relational schema after mapping the ER model to tables. Each box represents a table, and the arrows show foreign key references from child to parent.

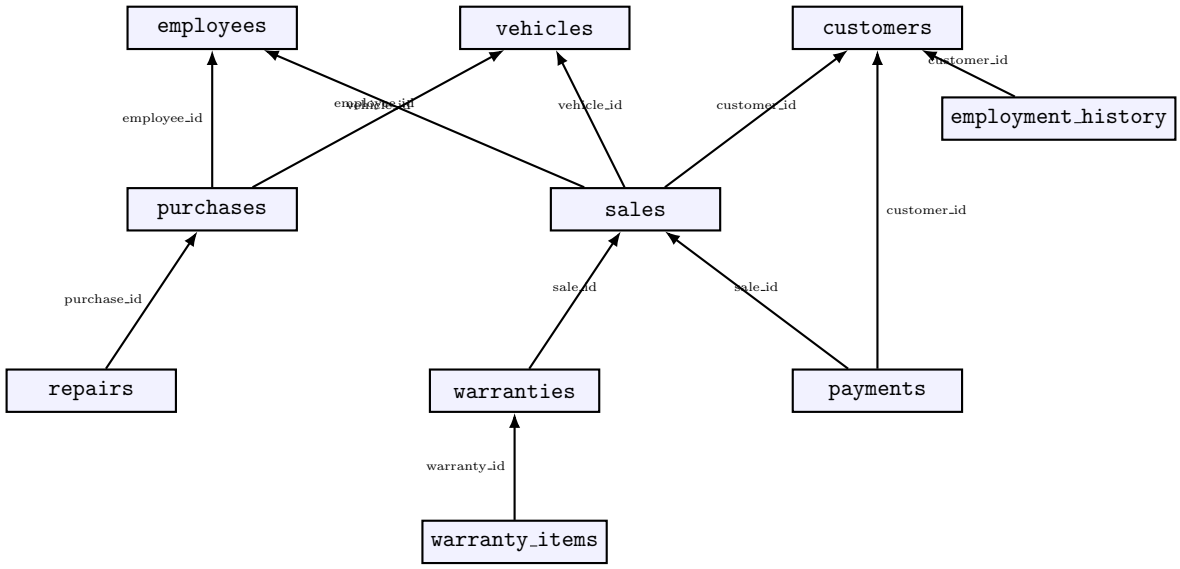


Figure 2: Relational Schema with Foreign Key Relationships

3.4 Normalization

We went through normalization in class covering 1NF, 2NF, 3NF, and BCNF. All of my tables are in 3NF. There are no partial dependencies since every primary key is a single-column AUTO_INCREMENT integer, which means 2NF is satisfied automatically. There are no transitive dependencies either: each non-key attribute depends directly on the primary key of its table, not on some other non-key column.

That said, I did make two deliberate denormalizations. The first is `num_late_payments` and `avg_days_late` on the customers table. Technically I could calculate these from the payments table every time, but the payment form spec required showing them. Recalculating with an aggregate query on every page load seemed slow and unnecessary, so I stored them directly on the customer record. It is a trade-off between strict normalization and performance.

The second is `customer_id` on the payments table. That info already exists on the sales table (which payments link to through `sale_id`), so it is redundant. But having it directly on payments means I do not need to JOIN through sales every time I want to look up a customer's payment history. It is another deliberate choice for query simplicity.

BCNF is stricter than 3NF, but for this project 3NF was the right target. The two denormalizations are documented and intentional.

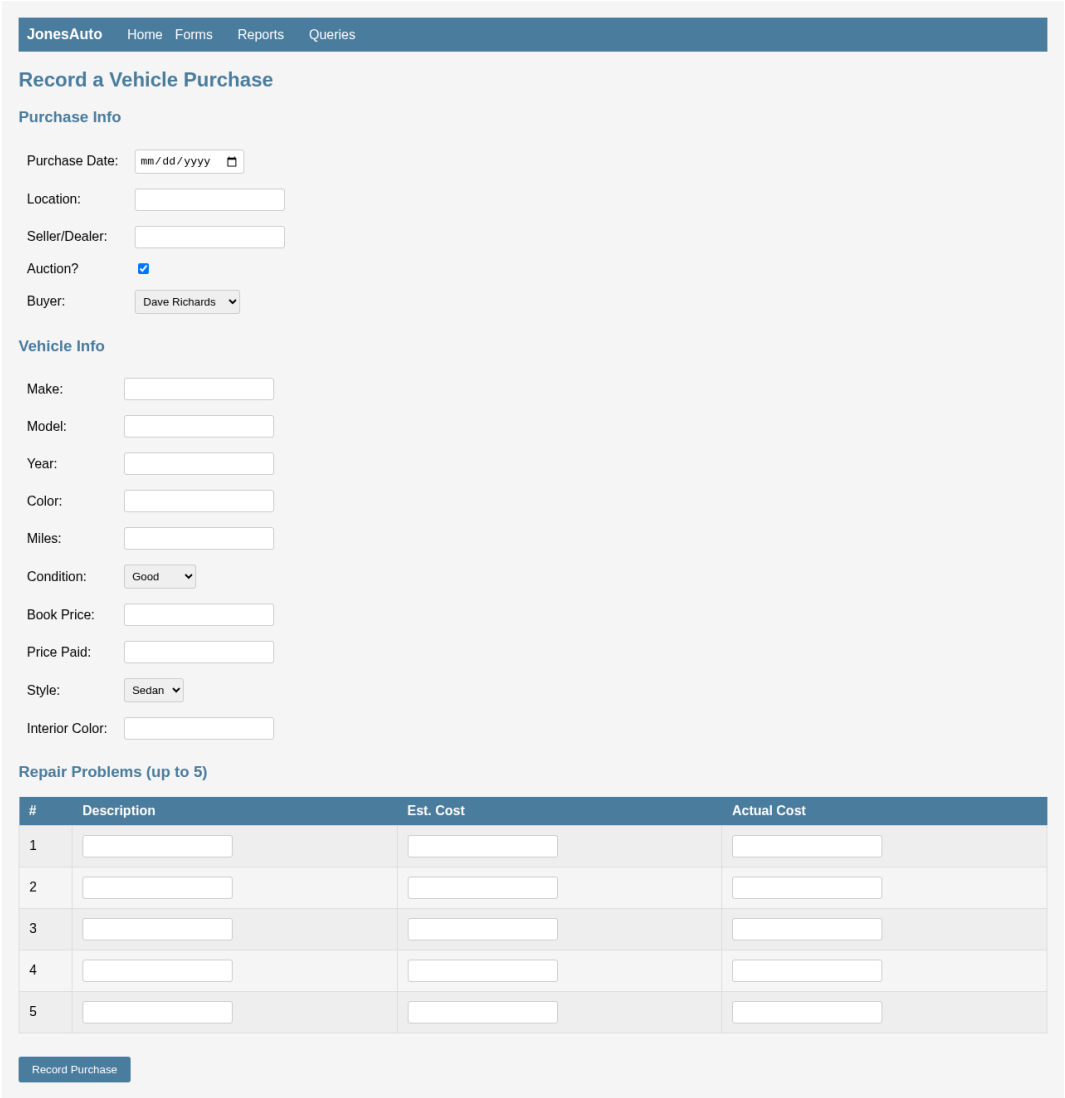
4 Stage 3: Forms and Reports

Stage 3 was where the bulk of the coding happened. I built 6 input forms and 6 output reports, all in PHP with mysqli for the database calls. The forms follow the two-file pattern from the class tutorials: an HTML form collects the input, and a PHP processor script handles the INSERT.

4.1 Input Forms

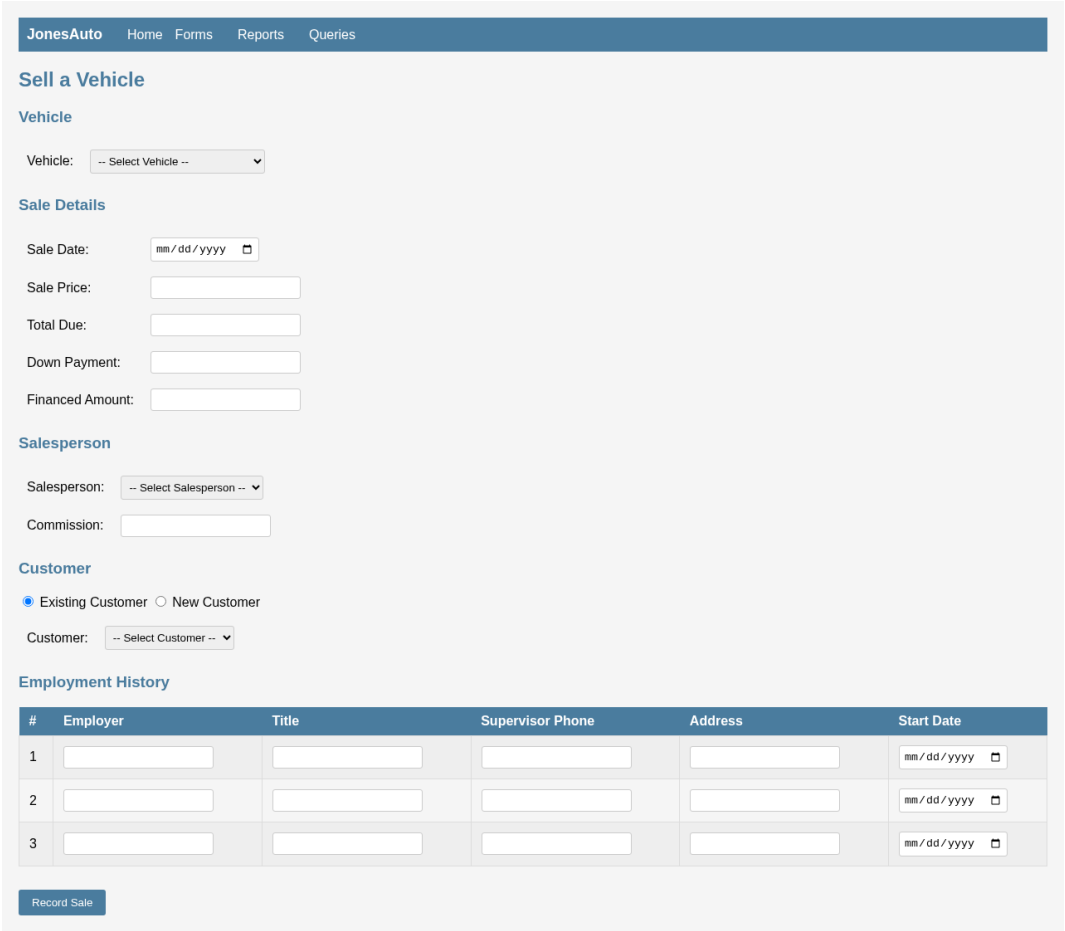
Purchase Form (`purchase_form.php`)

This form records a car bought at auction. The top section captures purchase info: date, location, seller/dealer name, and a checkbox for whether it was an auction purchase. The buyer employee gets selected from a dropdown populated from the employees table, filtered to only show people with the buyer role. The middle section is the vehicle details: make, model, year, color, miles, condition (dropdown with Excellent/Good/Fair/Poor), book price, style (Sedan/SUV/Truck/Van/Coupe), interior color, and price paid. The bottom section has up to 5 repair rows where you enter a problem description, estimated cost, and actual cost. Submitting the form inserts into three tables: vehicles, purchases, and repairs.



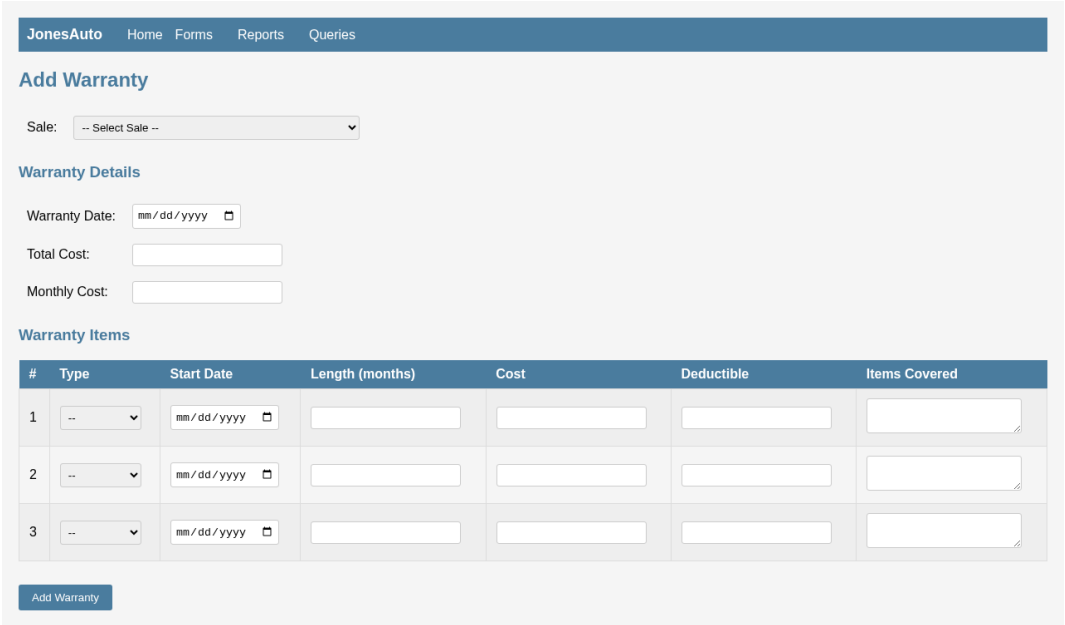
Sale Form (`sale_form.php`)

This one records a car sale. There is a toggle at the top for choosing between an existing customer and a new one. If you pick "New Customer," a set of fields appears for entering their info: name, phone, address, city, state, zip, gender, and date of birth. If you pick "Existing Customer," you just select from a dropdown. The vehicle is selected from a list of available cars. Then there are the sale details: date, sale price, total due, down payment, financed amount, and commission. At the bottom there is space for up to 3 employment history entries for the customer. The tricky part was the existing/new customer toggle since it changes which fields show up on the page. Submitting inserts into sales and employment_history, creates a new customer record if needed, and updates the vehicle status to sold.



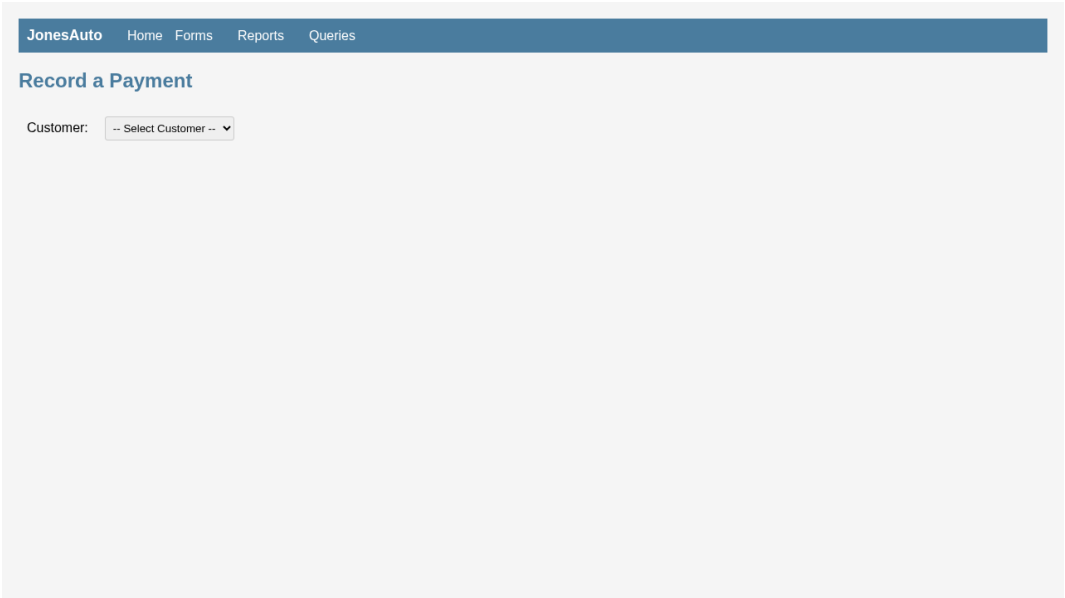
Warranty Form (`warranty_form.php`)

This form adds warranty coverage to an existing sale. You pick a sale from a dropdown that shows the customer name, vehicle info, and sale date. When you select a sale, it fills in the vehicle, customer, and employee IDs using hidden fields and a bit of JavaScript. Then you enter the overall warranty details: date, total cost, and monthly cost. Below that there are up to 3 warranty item rows, each with a type dropdown (Drive-Train, Exterior, Interior, Electrical), start date, length in months, cost, deductible, and a text area for items covered. Submitting inserts into the warranties and warranty_items tables.



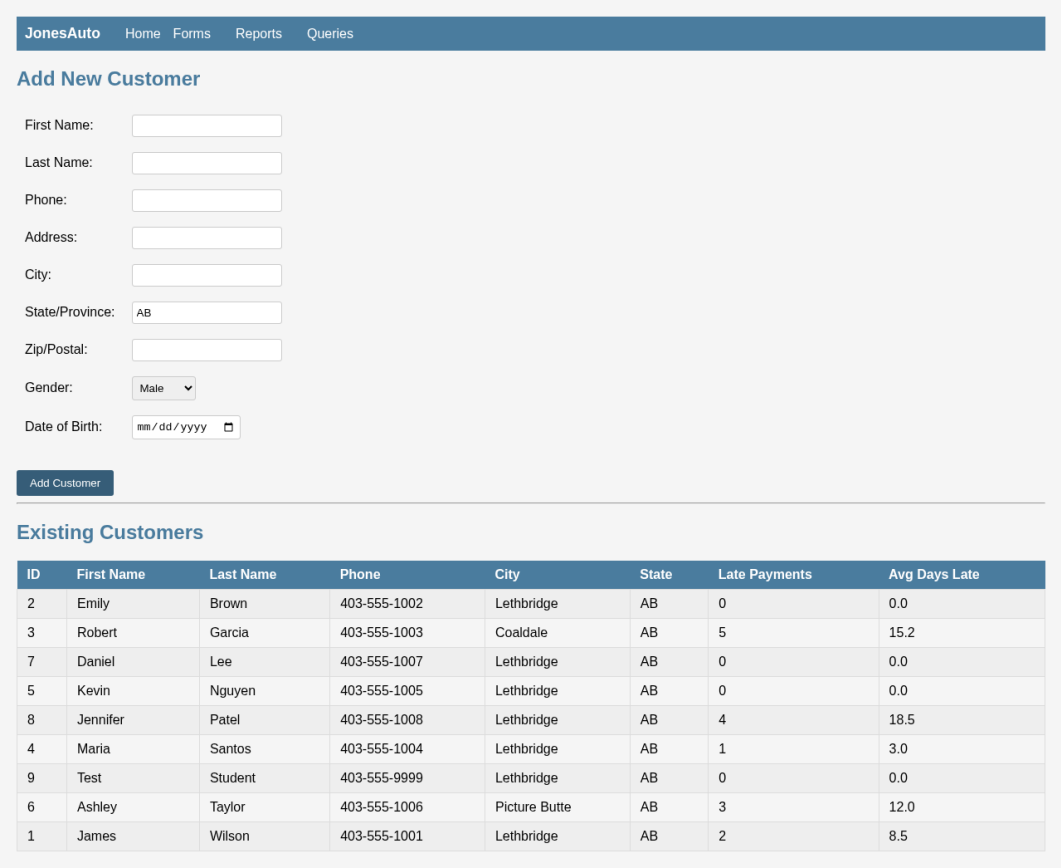
Payment Form (`payment_form.php`)

This form records a customer payment. There is a customer dropdown at the top that reloads the page when you select someone, so it can show that customer's info and existing payments below. Once a customer is selected, you see their gender, date of birth, number of late payments, and average days late. Then you pick which sale the payment is for, and enter the payment date, due date, paid date, amount, and bank account. If the paid date is after the due date, it gets flagged as late and the customer's late payment stats get recalculated. Below the form there is a table showing all existing payments for that customer, with late ones highlighted in red.



Customer Form (`customer_form.php`)

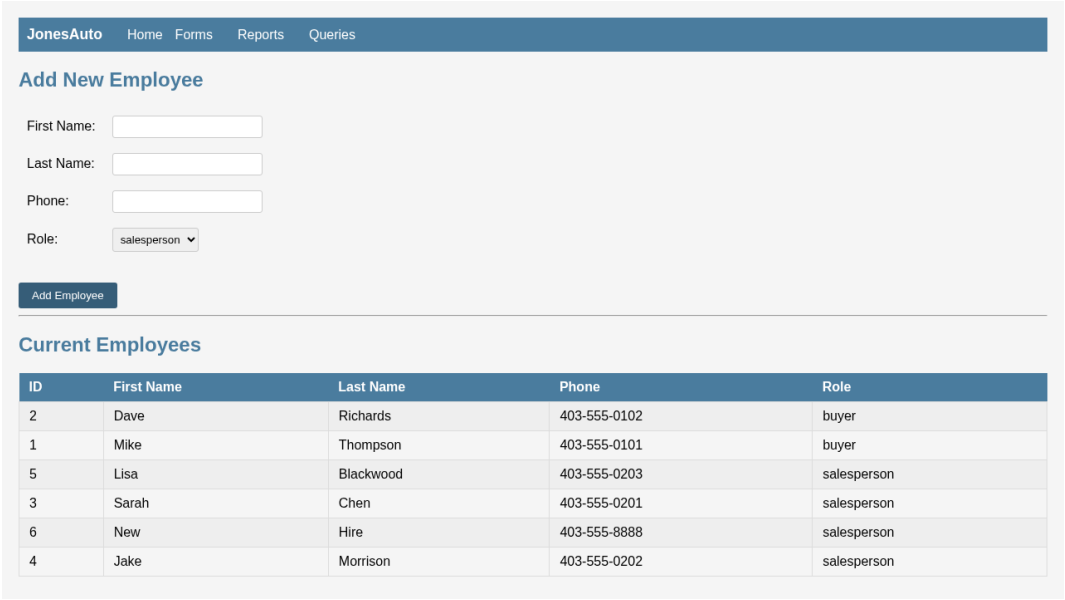
This is a simple add-and-view form. The top half has input fields for first name, last name, phone, address, city, state/province (defaults to AB), zip/postal code, gender, and date of birth. When you submit, it inserts into the customers table. Below the form there is a table showing all existing customers with their contact info, late payment count, and average days late.



ID	First Name	Last Name	Phone	City	State	Late Payments	Avg Days Late
2	Emily	Brown	403-555-1002	Lethbridge	AB	0	0.0
3	Robert	Garcia	403-555-1003	Coaldale	AB	5	15.2
7	Daniel	Lee	403-555-1007	Lethbridge	AB	0	0.0
5	Kevin	Nguyen	403-555-1005	Lethbridge	AB	0	0.0
8	Jennifer	Patel	403-555-1008	Lethbridge	AB	4	18.5
4	Maria	Santos	403-555-1004	Lethbridge	AB	1	3.0
9	Test	Student	403-555-9999	Lethbridge	AB	0	0.0
6	Ashley	Taylor	403-555-1006	Picture Butte	AB	3	12.0
1	James	Wilson	403-555-1001	Lethbridge	AB	2	8.5

Employee Form (`employee_form.php`)

Same pattern as the customer form. The input section has fields for first name, last name, phone, and a role dropdown (salesperson, buyer, or both). Existing employees are shown in a table below with their ID, name, phone, and role. Inserts into the employees table on submit.



4.2 Output Reports

The reports are read-only pages that pull data from the database and display it in HTML tables. Most of them have some kind of filter so you can narrow down what you are looking at.

Current Inventory (`inventory.php`)

Shows all vehicles with status "available." There is a text input at the top for filtering by make. For each car it shows make, model, year, color, miles, condition, book price, price paid, and total repair costs summed from the repairs table. It also calculates a total cost column (price paid plus repairs). The query uses a JOIN between vehicles and purchases, plus a subquery on repairs.

JonesAuto Home Forms Reports Queries

Current Inventory

Filter by Make: Filter

Showing: All Makes

Make	Model	Year	Color	Miles	Condition	Book Price	Price Paid	Repair Costs	Total Cost
Chevrolet	Malibu	2017	Black	110,000	Fair	$12,500.00	$8,200.00	$270.00	$8,470.00
Ford	Focus	2016	Blue	135,000	Fair	$7,200.00	$4,500.00	$380.00	$4,880.00
Honda	Civic	2020	Blue	45,000	Good	$17,200.00	$13,500.00	$0.00	$13,500.00
Honda	CR-V	2019	Black	71,000	Good	$22,500.00	$17,800.00	$170.00	$17,970.00
Hyundai	Elantra	2018	White	88,000	Fair	$11,800.00	$7,500.00	$625.00	$8,125.00
Kia	Forte	2021	Silver	29,000	Excellent	$16,200.00	$12,800.00	$0.00	$12,800.00
Mazda	3	2019	Red	58,000	Good	$16,500.00	$12,500.00	$310.00	$12,810.00
Nissan	Altima	2019	Grey	67,000	Good	$15,900.00	$11,800.00	$0.00	$11,800.00
Subaru	Outback	2018	Green	93,000	Good	$19,800.00	$15,500.00	$280.00	$15,780.00
Toyota	Camry	2019	Silver	78,000	Good	$18,500.00	$14,200.00	$470.00	$14,670.00
Toyota	Tacoma	2020	White	47,000	Good	$30,200.00	$24,800.00	$95.00	$24,895.00

11 vehicle(s) found.

Sales Summary (`sales_report.php`)

Shows all completed sales. You can filter by date range using a from/to date picker. For each sale it shows the date, vehicle info, customer name, salesperson, sale price, purchase cost, repair costs, and profit. The profit calculation was one of the trickier queries since it pulls from vehicles, purchases, repairs, sales, customers, and employees. Profit numbers are color-coded green for positive and red for negative. There is a totals row at the bottom.

JonesAuto Home Forms Reports Queries

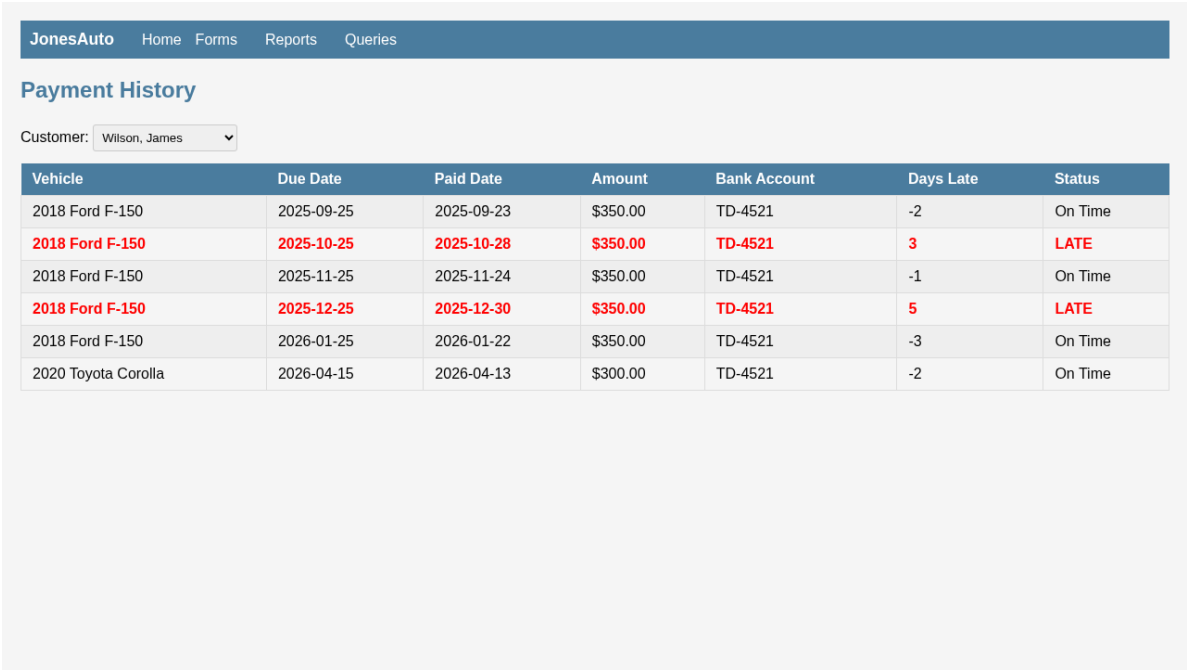
Sales Report

From: mm/dd/yyyy To: mm/dd/yyyy Filter

Date	Vehicle	Customer	Salesperson	Sale Price	Cost	Repairs	Profit
2026-03-15	2020 Toyota Corolla	James Wilson	Jake Morrison	$19,500.00	$14,200.00	$0.00	$5,300.00
2025-10-25	2019 Dodge Ram 1500	Kevin Nguyen	Jake Morrison	$32,500.00	$22,000.00	$280.00	$10,220.00
2025-10-15	2021 Toyota RAV4	Emily Brown	Jake Morrison	$29,500.00	$21,500.00	$90.00	$7,910.00
2025-09-30	2017 Honda Accord	Ashley Taylor	Sarah Chen	$16,800.00	$10,200.00	$190.00	$6,410.00
2025-09-20	2020 Ford Escape	Robert Garcia	Sarah Chen	$22,800.00	$15,200.00	$470.00	$7,130.00
2025-08-25	2018 Ford F-150	James Wilson	Sarah Chen	$26,500.00	$18,500.00	$710.00	$7,290.00
2025-08-10	2017 Chevrolet Cruze	Maria Santos	Lisa Blackwood	$11,200.00	$5,200.00	$675.00	$5,325.00
Totals				**$158,800.00**	**$106,800.00**	**$2,415.00**	**$49,585.00**

Payment History (`payment_history.php`)

You select a customer from a dropdown, and it shows all their payments in a table. The dropdown auto-submits when you change the selection. Each row shows the vehicle, due date, paid date, amount, bank account, days late, and status. Late payments (where `paid_date` is after `due_date`) get a red highlight using a CSS class. The query JOINs payments, sales, and vehicles.



Vehicle	Due Date	Paid Date	Amount	Bank Account	Days Late	Status
2018 Ford F-150	2025-09-25	2025-09-23	$350.00	TD-4521	-2	On Time
2018 Ford F-150	**2025-10-25**	**2025-10-28**	**$350.00**	**TD-4521**	**3**	**LATE**
2018 Ford F-150	2025-11-25	2025-11-24	$350.00	TD-4521	-1	On Time
2018 Ford F-150	**2025-12-25**	**2025-12-30**	**$350.00**	**TD-4521**	**5**	**LATE**
2018 Ford F-150	2026-01-25	2026-01-22	$350.00	TD-4521	-3	On Time
2020 Toyota Corolla	2026-04-15	2026-04-13	$300.00	TD-4521	-2	On Time

Repair Costs (`repair_summary.php`)

Shows estimated vs actual repair costs for every repair in the system. Each row has the vehicle info, purchase date, problem number, description, estimated cost, actual cost, and the difference. Rows where the actual cost exceeded the estimate get highlighted. This helps the dealership see which purchases ended up costing more to fix than they expected.

JonesAuto Home Forms Reports Queries

Repair Cost Summary

Vehicle	Purchase Date	Problem #	Description	Est. Cost	Actual Cost	Difference
2019 Toyota Camry	2025-08-15	1	Scratch on rear bumper	$150.00	$120.00	$-30.00
2019 Toyota Camry	2025-08-15	2	Dent on driver door	$300.00	$350.00	+$50.00
2018 Ford F-150	2025-07-20	1	Cracked windshield	$400.00	$380.00	$-20.00
2018 Ford F-150	2025-07-20	2	Rust spot on wheel well	$200.00	$250.00	+$50.00
2018 Ford F-150	2025-07-20	3	Interior stain on back seat	$100.00	$80.00	$-20.00
2017 Chevrolet Malibu	2025-10-10	1	Paint chip on hood	$150.00	$160.00	+$10.00
2017 Chevrolet Malibu	2025-10-10	2	Broken side mirror	$120.00	$110.00	$-10.00
2021 Toyota RAV4	2025-09-25	1	Minor scratch on fender	$100.00	$90.00	$-10.00
2020 Ford Escape	2025-08-30	1	Dent on rear quarter panel	$250.00	$280.00	+$30.00
2020 Ford Escape	2025-08-30	2	Worn brake pads	$200.00	$190.00	$-10.00
2018 Hyundai Elantra	2025-10-28	1	Broken tail light	$80.00	$75.00	$-5.00
2018 Hyundai Elantra	2025-10-28	2	Seat tear on passenger side	$150.00	$200.00	+$50.00
2018 Hyundai Elantra	2025-10-28	3	AC not blowing cold	$300.00	$350.00	+$50.00
2019 Honda CR-V	2025-11-15	1	Scratch on passenger door	$180.00	$170.00	$-10.00
2017 Chevrolet Cruze	2025-07-05	1	Multiple body dents	$500.00	$620.00	+$120.00
2017 Chevrolet Cruze	2025-07-05	2	Headlight foggy	$60.00	$55.00	$-5.00
2016 Ford Focus	2025-11-20	1	Bumper crack	$200.00	$180.00	$-20.00
2016 Ford Focus	2025-11-20	2	Power window not working	$150.00	$200.00	+$50.00
2019 Dodge Ram 1500	2025-09-18	1	Rust on undercarriage	$300.00	$280.00	$-20.00
2018 Subaru Outback	2025-10-05	1	Paint fading on roof	$250.00	$240.00	$-10.00
2018 Subaru Outback	2025-10-05	2	Door handle loose	$50.00	$40.00	$-10.00
2017 Honda Accord	2025-08-12	1	Dent on trunk	$200.00	$190.00	$-10.00
2020 Toyota Tacoma	2026-04-01	1	Chip in windshield	$100.00	$95.00	$-5.00
2019 Mazda 3	2026-04-03	1	Scratch on hood	$120.00	$130.00	+$10.00
2019 Mazda 3	2026-04-03	2	Interior needs detailing	$200.00	$180.00	$-20.00

Active Warranties (`warranty_report.php`)

Lists all warranty items with their expiry info. The page color-codes rows by urgency: red background for expired warranties, yellow background for ones expiring within 30 days, and no color for ones with more time left. Each row shows customer name, vehicle, warranty type, start date, length in months, expiry date, days remaining, cost, deductible, and items covered. The query JOINs warranties, warranty_items, vehicles, and customers.

JonesAuto Home Forms Reports Queries

Active Warranties

Red = Expired | Yellow = Expiring within 30 days

Customer	Vehicle	Type	Start Date	Length	Expiry Date	Days Remaining	Cost	Deductible	Items Covered
Robert Garcia	2020 Ford Escape	Drive-Train	2025-09-20	7 months	2026-04-20	14	$900.00	$100.00	Engine, transmission, differential
James Wilson	2018 Ford F-150	Drive-Train	2025-08-25	8 months	2026-04-25	19	$800.00	$100.00	Engine, transmission, differential, drive shaft
James Wilson	2018 Ford F-150	Exterior	2025-08-25	9 months	2026-05-25	49	$400.00	$50.00	Paint, body panels, bumpers
Emily Brown	2021 Toyota RAV4	Exterior	2025-10-15	12 months	2026-10-15	192	$600.00	$50.00	Paint, body panels, bumpers, mirrors
Kevin Nguyen	2019 Dodge Ram 1500	Interior	2025-10-25	12 months	2026-10-25	202	$400.00	$100.00	Seats, dashboard, carpet, headliner
Emily Brown	2021 Toyota RAV4	Drive-Train	2025-10-15	24 months	2027-10-15	557	$1,200.00	$75.00	Engine, transmission, differential, drive shaft, CV joints
Kevin Nguyen	2019 Dodge Ram 1500	Drive-Train	2025-10-25	24 months	2027-10-25	567	$1,500.00	$50.00	Engine, transmission, differential, drive shaft, transfer case
Kevin Nguyen	2019 Dodge Ram 1500	Exterior	2025-10-25	24 months	2027-10-25	567	$500.00	$75.00	Paint, body panels

Late Payments (`late_payments.php`)

Shows all customers who have at least one late payment, sorted worst-first by late count. Each row has the customer name, phone, total payments, number of late payments, and average days late. It uses a GROUP BY on customer with a HAVING clause to only show customers where `late_count` is greater than zero. This is basically a credit risk report for the dealership.

JonesAuto Home Forms Reports Queries

Late Payment Customers

Customer	Phone	Total Payments	Late Payments	Avg Days Late
Robert Garcia	403-555-1003	5	5	15.2
Ashley Taylor	403-555-1006	4	3	12.0
James Wilson	403-555-1001	6	2	8.5
Maria Santos	403-555-1004	4	1	3.0

5 Stage 4: Business Queries

The project spec required at least 10 queries with at least 5 being advanced (multi-table JOINs or subqueries). I ended up with 12 total: 5 simple and 7 advanced. They are all on a single queries page with a dropdown selector. You pick a query, hit submit, and the results show up in a table below.

For Query 3 there is also a text input so you can search by make. The SQL for each query is shown on the page along with the results, which is helpful for debugging and for the demo.

5.1 Query 1: What cars do we have on the lot right now? (Simple)

This one pulls up every vehicle that is still available for sale. It is the quickest way for a salesperson to see what is on the lot without walking outside.

```
SELECT * FROM vehicles
WHERE status = 'available'
ORDER BY make, model
```

SQL features: basic SELECT with WHERE, ORDER BY

5.2 Query 2: Show me all our customers (Simple)

Lists every customer in the system with their contact info. Useful for the office staff when they need to look someone up or check a phone number.

```
SELECT customer_id, first_name, last_name, phone, city, state
FROM customers
ORDER BY last_name
```

SQL features: basic SELECT with ORDER BY

5.3 Query 3: Do we have any [make] cars in stock? (Simple)

This one takes a text input for the make, so the actual query uses the value the user types in. It is for when a customer calls and asks "do you have any Toyotas?" and you need a quick answer.

```
SELECT * FROM vehicles
WHERE make = '[make]' AND status = 'available'
```

SQL features: WHERE with parameter, AND condition

5.4 Query 4: Who are our employees? (Simple)

Shows all employees sorted by their role and last name. Pretty straightforward, just a quick reference for who works here and what they do.

```
SELECT * FROM employees
ORDER BY role, last_name
```

SQL features: basic SELECT with ORDER BY

5.5 Query 5: How many cars did we buy this month? (Simple)

Gives a count of purchases made in the current month along with the total amount spent. The MONTH() and YEAR() functions with CURDATE() make it automatically filter to the current month.

```
SELECT COUNT(*) as cars_bought, SUM(price_paid) as total_spent
FROM purchases
WHERE MONTH(purchase_date) = MONTH(CURDATE())
AND YEAR(purchase_date) = YEAR(CURDATE())
```

SQL features: COUNT, SUM aggregates, MONTH/YEAR functions with CURDATE()

5.6 Query 6: Which cars did we buy below book price? (Advanced)

This one is useful for the buyers to see which deals paid off. It joins vehicles and purchases to compare what was paid against the book price, then calculates the savings.

```
SELECT v.make, v.model, v.year, v.book_price, p.price_paid,
       (v.book_price - p.price_paid) as savings
FROM vehicles v
JOIN purchases p ON v.vehicle_id = p.vehicle_id
WHERE p.price_paid < v.book_price
ORDER BY savings DESC
```

SQL features: JOIN between vehicles and purchases, calculated column (savings)

5.7 Query 7: What is our profit on each sale? (Advanced)

This was the hardest query to write. The profit calculation needs to factor in the purchase price AND all repair costs, so there is a correlated subquery that sums repairs for each purchase. The COALESCE handles cases where a vehicle had no repairs so the sum does not come back as NULL.

```
SELECT v.make, v.model, v.year, s.sale_price, p.price_paid,
       (SELECT COALESCE(SUM(r.actual_cost),0) FROM repairs r
        WHERE r.purchase_id = p.purchase_id) as repair_costs,
       (s.sale_price - p.price_paid -
        (SELECT COALESCE(SUM(r.actual_cost),0) FROM repairs r
         WHERE r.purchase_id = p.purchase_id)) as profit
```

```
FROM sales s
JOIN vehicles v ON s.vehicle_id = v.vehicle_id
JOIN purchases p ON v.vehicle_id = p.vehicle_id
ORDER BY profit DESC
```

SQL features: correlated subquery for repair costs, multiple JOINs (sales + vehicles + purchases), COALESCE, calculated profit

5.8 Query 8: Which customers have late payments? (Advanced)

Shows every late payment along with the customer name, vehicle info, and how many days late it was. The DATEDIFF function calculates how many days late each payment was.

```
SELECT DISTINCT CONCAT(c.first_name, ' ', c.last_name) as
   customer_name,
       c.phone, v.make, v.model, v.year,
       DATEDIFF(pay.paid_date, pay.due_date) as days_late
FROM payments pay
JOIN customers c ON pay.customer_id = c.customer_id
JOIN sales s ON pay.sale_id = s.sale_id
JOIN vehicles v ON s.vehicle_id = v.vehicle_id
WHERE pay.paid_date > pay.due_date
ORDER BY days_late DESC
```

SQL features: 4-table JOIN (payments + customers + sales + vehicles), DATEDIFF, DISTINCT, WHERE on date comparison

5.9 Query 9: Who is our top salesperson? (Advanced)

Groups sales by employee and shows their total revenue and commission. The COUNT gives number of sales and SUM gives the dollar totals. Sorted by number of sales so the top performer shows up first.

```
SELECT CONCAT(e.first_name, ' ', e.last_name) as salesperson,
       COUNT(s.sale_id) as num_sales,
       SUM(s.sale_price) as total_revenue,
       SUM(s.commission) as total_commission
FROM sales s
JOIN employees e ON s.employee_id = e.employee_id
GROUP BY e.employee_id
ORDER BY num_sales DESC
```

SQL features: JOIN, GROUP BY, COUNT, SUM aggregates

5.10 Query 10: Which warranties expire within 90 days? (Advanced)

This uses DATE_ADD to calculate the expiry date from start_date + length_months, then BETWEEN to check if it falls within the next 90 days. The DATEDIFF gives the exact number of days remaining so the dealership can follow up with customers before their coverage runs out.

```
SELECT CONCAT(c.first_name, ' ', c.last_name) as customer_name,
       v.make, v.model, v.year,
       wi.warranty_type, wi.start_date,
       DATE_ADD(wi.start_date, INTERVAL wi.length_months MONTH)
           as expiry_date,
       DATEDIFF(DATE_ADD(wi.start_date,
           INTERVAL wi.length_months MONTH), CURDATE())
           as days_remaining
FROM warranty_items wi
JOIN warranties w ON wi.warranty_id = w.warranty_id
JOIN vehicles v ON w.vehicle_id = v.vehicle_id
JOIN customers c ON w.customer_id = c.customer_id
WHERE DATE_ADD(wi.start_date, INTERVAL wi.length_months MONTH)
      BETWEEN CURDATE() AND DATE_ADD(CURDATE(), INTERVAL 90 DAY)
ORDER BY expiry_date
```

SQL features: DATE_ADD, DATEDIFF, BETWEEN with date range, 4-table JOIN

5.11 Query 11: Where did repair costs go over budget? (Advanced)

The HAVING clause filters groups to only show vehicles where actual repair costs exceeded estimates. The difference between WHERE and HAVING is that WHERE filters individual rows before grouping, HAVING filters groups after. This query groups repairs by purchase and compares the totals.

```
SELECT v.make, v.model, v.year,
       SUM(r.est_cost) as estimated_total,
       SUM(r.actual_cost) as actual_total,
       (SUM(r.actual_cost) - SUM(r.est_cost)) as over_budget
FROM repairs r
JOIN purchases p ON r.purchase_id = p.purchase_id
JOIN vehicles v ON p.vehicle_id = v.vehicle_id
GROUP BY p.purchase_id
HAVING SUM(r.actual_cost) > SUM(r.est_cost)
ORDER BY over_budget DESC
```

SQL features: GROUP BY with HAVING, SUM aggregates, calculated column

5.12 Query 12: Which customers bought more than one car? (Advanced)

Simple but useful. `HAVING COUNT(s.sale_id) > 1` keeps only repeat buyers. This is good for identifying loyal customers who might be interested in future deals.

```
SELECT CONCAT(c.first_name, ' ', c.last_name) as customer_name,
       c.phone,
       COUNT(s.sale_id) as cars_bought,
       SUM(s.sale_price) as total_spent
FROM customers c
JOIN sales s ON c.customer_id = s.customer_id
GROUP BY c.customer_id
HAVING COUNT(s.sale_id) > 1
ORDER BY cars_bought DESC
```

SQL features: GROUP BY with HAVING COUNT, aggregate functions

6 Assumptions and Design Decisions

Here are the main assumptions I made and the reasoning behind the bigger design decisions.

1. **Used `condition_desc` instead of `condition`**: MySQL has `condition` as a reserved word, so I could not use it as a column name without backticks everywhere. Renaming to `condition_desc` was the simplest fix. Not ideal but clear enough.

2. **Denormalized late payment stats on customers table**: The `num_late_payments` and `avg_days_late` columns on the customers table are technically redundant since they could be calculated from the payments table each time. But the payment form spec required showing these values, and recalculating on every page load with an aggregate query would be slower. I chose to store them directly and update them whenever a late payment is recorded. Normalization trades some redundancy for query performance, and this is one of those trade-offs.

3. **`customer_id` duplicated on payments**: The payments table has `customer_id` even though you could get it by joining through sales. It is there so I can quickly look up all payments for a customer without an extra JOIN. Another deliberate denormalization.

4. **AUTO_INCREMENT surrogate keys everywhere**: Every table uses an INT AUTO_INCREMENT primary key instead of composite or natural keys. It is simpler for foreign key references and avoids issues if natural values change. I went with surrogate keys because they are straightforward.

5. **Tables created in FK dependency order**: The `db_setup.sql` file creates tables in the right order (employees and vehicles first, then purchases, etc.) so the foreign key constraints do not fail. This way the script runs straight through without needing deferred constraint checks.

6. **Alberta-specific sample data**: All the seed data uses Lethbridge, Calgary, and Medicine Hat locations with Alberta postal codes. Keeps it realistic for the course context.

7. **Deliberate late payment data in seed**: I included some payment records where `paid_date` is after `due_date` on purpose, so the late payment reports and queries actually have data to show during testing and the demo.

7 What I Learned

The biggest thing I took away from this project is how normalization actually works in practice. In class we walked through functional dependencies and decomposition on paper, but actually deciding where to denormalize for a real application was a different kind of challenge. It is one thing to identify a transitive dependency; it is another to decide whether the performance trade-off is worth it.

I had never used PHP before this course. The procedural mysqli pattern from the tutorials turned out to be pretty straightforward once I got the hang of it. The hardest part was debugging SQL errors since PHP just gives you a generic error string and you have to figure out which part of the query is wrong.

Drawing the ER diagram first and then mapping it to relational tables made the schema design way more structured than if I had just started creating tables. It forced me to think about relationships and cardinalities before writing any SQL.

Setting up the foreign keys caught several data integrity issues during testing. I tried inserting a sale for a `vehicle_id` that did not exist and MySQL blocked it. That is exactly what referential integrity is supposed to do, and it is way better than having orphaned records.

Writing the advanced queries (especially Query 7 with the correlated subquery) pushed me to actually understand how JOINs and subqueries work together. The examples from class used the banking database, but applying those patterns to a different domain helped it click.

8 Future Improvements

If I had more time or was building this for a real dealership, here is what I would add:

1. **User login system**: Right now anyone can access everything. A real system would need separate logins for buyers, salespeople, and managers, with different permissions for each.

2. **Search and sorting on reports**: The reports have basic filters but you cannot sort by clicking column headers or search within results. Adding JavaScript for client-side sorting would make the reports much more usable.

3. **Mobile/responsive layout**: The forms use HTML tables for layout, which does not work great on phones. The buyers at auctions mentioned wanting a portable version, so a responsive design would be a big improvement.

4. **Credit bureau integration**: The spec mentions that really bad customers get reported to the credit bureau. Right now there is no integration for that. An API connection to a credit reporting service would automate that process.

5. **Inventory alerts**: The dealership tries to keep no more than 50 cars on the lot. An alert system that warns when inventory is getting high (or when popular makes are running low) would help the buyers make better decisions at auctions.

Overall I am pretty happy with how the project turned out. It covers the full lifecycle from buying cars to collecting payments, and the queries answer real business questions that a dealership would actually care about.